Cognizant - UpGrad Training Program
WEEKLY CODING SUBMISSION

Name	**:**	**Durgesh Vijay Bhangale**
Week	**:**	**Week 6**
Batch	**:**	**B1_Python**
Trainer	**:**	**Kunal**

Day 1 (8-4-26)

A startup wants to build a Food Delivery Platform using microservices.

```
public class CustomerServiceTests {
   private readonly CustomerService _service;

   public CustomerServiceTests() {
      var options = new DbContextOptionsBuilder<AppDbContext>()
         .UseInMemoryDatabase("CustomerDb").Options;
      var context = new AppDbContext(options);
      _service = new CustomerService(context);
   }

   [Fact]
   public void CreateCustomer_ShouldReturnOk() {
      var result = _service.CreateCustomer(new Customer { Name="Durgesh", Email="d@test.com" });
      Assert.IsType<OkObjectResult>(result);
   }

   [Fact]
   public void GetCustomer_ShouldReturnCorrectData() {
      _service.CreateCustomer(new Customer { Name="Amit", Email="a@test.com" });
      var result = _service.GetCustomer(1) as OkObjectResult;
      Assert.NotNull(result);
      Assert.Equal("Amit", ((Customer)result.Value).Name);
   }

   [Fact]
   public void InvalidCustomer_ShouldReturnBadRequest() {
      var result = _service.CreateCustomer(new Customer { Name="", Email="" });
      Assert.IsType<BadRequestObjectResult>(result);
   }
}
```

[Fact]

RestaurantService.Tests

```
public void AddRestaurant_ShouldReturnSuccess() {
   var result = _service.AddRestaurant(new Restaurant { Name="FoodHub" });
   Assert.IsType<OkObjectResult>(result);
}

[Fact]
public void GetRestaurantById_ShouldReturnSuccess() {
   var res = _service.AddRestaurant(new Restaurant { Name="PizzaPlace" });
   var result = _service.GetRestaurant(1);
   Assert.IsType<OkObjectResult>(result);
}

[Fact]
public void InvalidId_ShouldReturnNotFound() {
   var result = _service.GetRestaurant(999);
   Assert.IsType<NotFoundResult>(result);
}
```

Menuservice.tests

```
[Fact]
public void AddMenuItem_ShouldReturnSuccess() {
   var result = _service.AddMenuItem(new MenuItem { Name="Burger", Price=100, RestaurantId=1
});
   Assert.IsType<OkObjectResult>(result);
}

[Fact]
public void GetMenuByRestaurantId_ShouldReturnSuccess() {
   _service.AddMenuItem(new MenuItem { Name="Pizza", Price=200, RestaurantId=1 });
   var result = _service.GetMenuByRestaurantId(1);
   Assert.IsType<OkObjectResult>(result);
}

[Fact]
public void InvalidPrice_ShouldReturnBadRequest() {
   var result = _service.AddMenuItem(new MenuItem { Name="Fries", Price=-10, RestaurantId=1 });
   Assert.IsType<BadRequestObjectResult>(result);
}
```

Orderservice.tests

```
[Fact]
public void CreateOrder_ShouldReturnOk() {
   var mockCustomer = new Mock<ICustomerService>();
   mockCustomer.Setup(c => c.Exists(It.IsAny<int>())).Returns(true);

   var mockRestaurant = new Mock<IRestaurantService>();
   mockRestaurant.Setup(r => r.Exists(It.IsAny<int>())).Returns(true);

   var mockMenu = new Mock<IMenuService>();
   mockMenu.Setup(m => m.Exists(It.IsAny<int>())).Returns(true);

   var service = new OrderService(mockCustomer.Object, mockRestaurant.Object,
mockMenu.Object);
   var result = service.CreateOrder(new Order { CustomerId=1, RestaurantId=1, MenuItemId=1 });

   Assert.IsType<OkObjectResult>(result);
}

[Fact]
public void CreateOrder_InvalidCustomer_ShouldReturnBadRequest() {
   var mockCustomer = new Mock<ICustomerService>();
   mockCustomer.Setup(c => c.Exists(It.IsAny<int>())).Returns(false);

   var service = new OrderService(mockCustomer.Object, new Mock<IRestaurantService>().Object,
new Mock<IMenuService>().Object);
   var result = service.CreateOrder(new Order { CustomerId=99 });

   Assert.IsType<BadRequestObjectResult>(result);
   Assert.Equal("Invalid Customer", ((BadRequestObjectResult)result).Value);
}
```

Day (9-4-26)

1. Create & List
mkdir test1 && cd test1
ls

2. Multiple Files
touch a.txt b.txt c.txt

3. Add Content
echo "Linux Practice" > a.txt

4. View Content
cat a.txt

5. Copy File
cp a.txt copy_a.txt

6. Rename File
mv b.txt new_b.txt

7. Delete File
rm c.txt

8. Move File
mkdir data
mv a.txt data/

9. Nested Directory
mkdir -p project/logs

10. List Recursively
ls -R

11. Check Permissions
ls -l

12. Make copy_a.txt executable
chmod +x copy_a.txt

13. Remove write permission
chmod -w copy_a.txt

14. Overwrite File
echo "Hello World" > file.txt

15. Append Data
echo "Second Line" >> file.txt

16. Use Pipe (filter Hello)
```
cat file.txt | grep "Hello"
```

17. Combine Commands (count lines)
```
wc -l < file.txt
```

18. Current User
```
whoami
```

19. System Date
```
date
```

20. Running Processes
```
ps aux
```

21. hello.sh
```
echo 'echo "Hello Linux"' > hello.sh
chmod +x hello.sh
./hello.sh
```

22. Username Script
```
echo 'whoami' > user.sh
chmod +x user.sh
./user.sh
```

23. Date Script
```
echo 'date' > date.sh
chmod +x date.sh
./date.sh
```

24. Variables
```
echo 'name="Durgesh"; echo $name' > var.sh
chmod +x var.sh
./var.sh
```

25. Conditional
```
echo 'if [ -f file.txt ]; then echo "Exists"; else echo "Not Found"; fi' > check.sh
chmod +x check.sh
./check.sh
```

26. Loop
```
echo 'for i in {1..5}; do echo $i; done' > loop.sh
chmod +x loop.sh
./loop.sh
```

27. Combine Script
```
echo 'echo "Hello $(whoami)"; echo "Today is $(date)"' > combo.sh
```

```
chmod +x combo.sh
./combo.sh
```

28. One go

```
touch f1.txt f2.txt f3.txt && \
echo "First" > f1.txt && echo "Second" > f2.txt && echo "Third" > f3.txt && \
mkdir folder && mv f1.txt folder/ && cat f2.txt
```

Day 3, 4(10-4-26)

1. Square and cube
```
num = int(input("Enter a number: "))
print("Square:", num**2, "Cube:", num**3)
```

2. First and last character
```
s = input("Enter a string: ")
print("First:", s[0], "Last:", s[-1])
```

3. Numbers divisible by 3 (1–20)
```
print([i for i in range(1, 21) if i % 3 == 0])
```

4. Largest of 3 numbers
```
a, b, c = map(int, input("Enter 3 numbers: ").split())
print("Largest:", max(a, b, c))
```

5. Function check > 100
```
def check(num): return num > 100
print(check(150))
```

6. Welcome message
```
name = input("Enter name: ")
print("Welcome Admin" if name.lower() == "admin" else "Welcome User")
```

7. Length without len()
```
lst = [1, 2, 3, 4]
count = 0
for _ in lst: count += 1
print("Length:", count)
```

8. Multiplication table
```
n = int(input("Enter number: "))
for i in range(1, 11): print(f"{n} x {i} = {n*i}")
```

9. Count > 10
```
lst = [5, 12, 18, 3, 25]
print(sum(1 for x in lst if x > 10))
```

10. Print word 5 times
```
word = input("Enter word: ")
for _ in range(5): print(word)
```

11. Odd numbers list
```
lst = [1, 2, 3, 4, 5, 6]
print([x for x in lst if x % 2 != 0])
```

12. Prime check
```
n = int(input("Enter number: "))
```

```
print("Prime" if all(n % i for i in range(2, int(n**0.5)+1)) and n > 1 else "Not Prime")
```

13. Second largest

```
lst = [10, 20, 4, 45, 99]
print(sorted(set(lst))[-2])
```

14. Count vowels/consonants

```
sentence = input("Enter sentence: ").lower()
vowels = "aeiou"
v = sum(1 for c in sentence if c in vowels)
c = sum(1 for c in sentence if c.isalpha() and c not in vowels)
print("Vowels:", v, "Consonants:", c)
```

15. Dictionary cubes

```
d = {i: i**3 for i in range(1, 6)}
print(d)
```

16. Replace spaces

```
s = "hello world python"
print(s.replace(" ", "_"))
```

17. Random numbers

```
import random
nums = [random.randint(1, 100) for _ in range(5)]
print(nums)
```

18. Count occurrences

```
lst = [1, 2, 2, 3, 4, 2]
num = 2
print(lst.count(num))
```

19. Print lines with "Python"

```
with open("sample.txt") as f:
    for line in f:
        if "Python" in line: print(line.strip())
```

20. Exception handling

```
try:
    n = int(input("Enter number: "))
    print("Square:", n**2)
except ValueError:
    print("Error: Please enter a valid number!")
```

21. Anagram check

```
s1, s2 = "listen", "silent"
print(sorted(s1) == sorted(s2))
```

22. Flatten list

```
lst = [1, [2,3], [4,5]]
flat = []
for i in lst:
    flat.extend(i if isinstance(i, list) else [i])
print(flat)
```

23. Dict from two lists

```
keys = ["a","b","c"]
values = [1,2,3]
print(dict(zip(keys, values)))
```

24. Find duplicates

```
lst = [1,2,3,2,4,5,1]
dupes = set([x for x in lst if lst.count(x) > 1])
print(dupes)
```

25. Sort without sort()

```
lst = [5,2,9,1]
for i in range(len(lst)):
    for j in range(i+1, len(lst)):
        if lst[i] > lst[j]:
            lst[i], lst[j] = lst[j], lst[i]
print(lst)
```

26. Reverse each word

```
sentence = "hello world"
print(" ".join([w[::-1] for w in sentence.split()]))
```

27. Lambda even/odd

```
check_even = lambda x: "Even" if x % 2 == 0 else "Odd"
print(check_even(7))
```

28. Read CSV age > 25

```
import csv
with open("data.csv") as f:
    reader = csv.DictReader(f)
    for row in reader:
        if int(row["age"]) > 25: print(row)
```

29. Nested dictionary values

```
d = {"a":{"x":1}, "b":{"y":2}}
for outer in d.values():
    for val in outer.values():
        print(val)
```

30. Separate even/odd

```
lst = [1,2,3,4,5,6]
evens = [x for x in lst if x % 2 == 0]
odds = [x for x in lst if x % 2 != 0]
print("Evens:", evens, "Odds:", odds)
```

Day 2

1. Vehicle → Car

```
class Vehicle:
    def __init__(self, brand, speed):
        self.brand = brand
        self.speed = speed
    def move(self):
        return "Vehicle is moving"

class Car(Vehicle):
    def move(self):
        return "Car is moving fast"

c = Car("Toyota", 120)
print(c.move())
```

2. Employee → Manager

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
    def display_details(self):
        return f"Name: {self.name}, Salary: {self.salary}"

class Manager(Employee):
    def __init__(self, name, salary, department):
        super().__init__(name, salary)
        self.department = department
    def display_details(self):
        return f"Name: {self.name}, Salary: {self.salary}, Department: {self.department}"

m = Manager("Durgesh", 50000, "IT")
print(m.display_details())
```

Shape → Rectangle, Circle

```
import math
class Shape:
    def area(self): return 0

class Rectangle(Shape):
    def __init__(self, l, w): self.l, self.w = l, w
    def area(self): return self.l * self.w
```

```
class Circle(Shape):
    def __init__(self, r): self.r = r
    def area(self): return math.pi * self.r**2

print(Rectangle(5, 3).area(), Circle(4).area())
```

BankAccount

```
class BankAccount:
    def __init__(self, acc, bal=0):
        self.account_number, self.balance = acc, bal
    def deposit(self, amt): self.balance += amt
    def withdraw(self, amt):
        if amt > self.balance: print("Insufficient balance")
        else: self.balance -= amt

acc = BankAccount("123", 1000)
acc.withdraw(1200)   validation
```

Student (Encapsulation)

```
class Student:
    def __init__(self): self.__marks = 0
    def set_marks(self, m):
        if 0 <= m <= 100: self.__marks = m
        else: print("Invalid marks")
    def get_marks(self): return self.__marks

s = Student(); s.set_marks(85); print(s.get_marks())
```

Book (__str__)

```
class Book:
    def __init__(self, title, author): self.title, self.author = title, author
    def __str__(self): return f"{self.title} by {self.author}"

print(Book("Python Basics", "Durgesh"))
```

CartItem (__repr__)

```
class CartItem:
    def __init__(self, name, price, qty): self.name, self.price, self.qty = name, price, qty
    def __repr__(self): return f"{self.name} - {self.qty} pcs @ ₹{self.price}"

print(CartItem("Laptop", 50000, 2))
```

8. Generator: squares

```
def squares(n):
    for i in range(1, n+1): yield i**2
print(list(squares(5)))
```

9. Generator: evens

```
def evens(n):
    for i in range(2, n+1, 2): yield i
print(list(evens(10)))
```

10. Custom iterator reverse

```
class ReverseList:
    def __init__(self, lst): self.lst, self.i = lst, len(lst)
    def __iter__(self): return self
    def __next__(self):
        if self.i == 0: raise StopIteration
        self.i -= 1; return self.lst[self.i]

for x in ReverseList([1,2,3]): print(x)
```

11. map → cubes

```
nums = [1,2,3,4]
print(list(map(lambda x: x**3, nums)))
```

12. filter → names length > 5

```
names = ["Durgesh","Amit","Komal","Tushar"]
print(list(filter(lambda n: len(n) > 5, names)))
```

13. reduce → product

```
from functools import reduce
lst = [2,3,4]
print(reduce(lambda x,y: x*y, lst))
```

14. Decorator → execution time

```
import time
def timer(func):
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        print("Execution time:", time.time()-start)
        return result
    return wrapper
```

```
@timer
def loop():
   for _ in range(1000000): pass
loop()
```

15. Decorator → admin check

```
def admin_only(func):
   def wrapper(user):
      if user == "admin": return func(user)
      else: print("Access Denied")
   return wrapper

@admin_only
def secret(user): print("Welcome Admin")
secret("admin"); secret("guest")
```

16. JSON → filter gmail

```
import json
data = [
   {"name":"Durgesh","email":"durgesh@gmail.com"},
   {"name":"Amit","email":"amit@yahoo.com"}
]
with open("users.json","w") as f: json.dump(data,f)

with open("users.json") as f:
   users = json.load(f)
   for u in users:
      if u["email"].endswith("@gmail.com"): print(u["email"])
```

17. Regex → extract emails

```
import re
text = "Contact: durgesh@gmail.com, amit@yahoo.com"
emails = re.findall(r"[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}", text)
print(emails)
```

<u>MINI PROJECT</u>

1. Smart Expense Tracker (CLI Based)

```
import json
import os
from datetime import datetime

FILE = "expenses.json"
def load_data():
    if os.path.exists(FILE):
        with open(FILE, "r") as f:
            return json.load(f)
    return []
def save_data(data):
    with open(FILE, "w") as f:
        json.dump(data, f, indent=4)
def add_expense(data):
    try:
        amount = float(input("Enter amount: "))
        category = input("Enter category (Food, Travel, etc.): ")
        date = input("Enter date (YYYY-MM-DD): ")
        datetime.strptime(date, "%Y-%m-%d")
        expense = {"amount": amount, "category": category, "date": date}
        data.append(expense)
        save_data(data)
        print("✅ Expense added successfully!")
    except ValueError:
        print("❌ Invalid input. Please try again.")
def view_expenses(data):
    if not data:
        print("No expenses found.")
        return
    print("\n--- All Expenses ---")
    for e in data:
        print(f"{e['date']} | {e['category']} | ₹{e['amount']}")

def filter_category(data):
    cat = input("Enter category to filter: ")
    filtered = [e for e in data if e['category'].lower() == cat.lower()]
    if not filtered:
        print("No expenses found in this category.")
    else:
        print(f"\n--- Expenses in {cat} ---")
        for e in filtered:
            print(f"{e['date']} | ₹{e['amount']}")
def monthly_summary(data):
    month = input("Enter month (YYYY-MM): ")
    total = sum(e['amount'] for e in data if e['date'].startswith(month))
    print(f"Total spending in {month}: ₹{total}")

def menu():
```

```
    data = load_data()
    while True:
        print("\n📊 Smart Expense Tracker")
        print("1. Add Expense")
        print("2. View All Expenses")
        print("3. Filter by Category")
        print("4. Monthly Summary")
        print("5. Exit")
        choice = input("Choose option: ")

        if choice == "1": add_expense(data)
        elif choice == "2": view_expenses(data)
        elif choice == "3": filter_category(data)
        elif choice == "4": monthly_summary(data)
        elif choice == "5":
            print("Goodbye! 👋")
            break
        else:
            print("Invalid choice. Try again.")

if __name__ == "__main__":
    menu()
```

2. Student Management System (OOP Based)

```
import json, os

FILE = "students.json"

class Student:
    def __init__(self, name, roll, marks):
        self.name, self.roll, self.marks = name, roll, marks
    def to_dict(self): return {"name": self.name, "roll": self.roll, "marks": self.marks}

def load(): return json.load(open(FILE)) if os.path.exists(FILE) else []
def save(data): json.dump(data, open(FILE,"w"), indent=4)

def add(data):
    name = input("Name: "); roll = input("Roll: "); marks = int(input("Marks: "))
    data.append(Student(name, roll, marks).to_dict()); save(data)

def view(data): [print(s) for s in data]
def search(data):
    roll = input("Enter roll: ")
    print(next((s for s in data if s["roll"]==roll), "Not found"))
def update(data):
    roll = input("Roll to update: ")
```

```
    for s in data:
        if s["roll"]==roll: s["marks"]=int(input("New marks: ")); save(data); return
    print("Not found")
def delete(data):
    roll = input("Roll to delete: ")
    data[:] = [s for s in data if s["roll"]!=roll]; save(data)

def menu():
    data = load()
    while True:
        print("\n1.Add 2.View 3.Search 4.Update 5.Delete 6.Exit")
        ch = input("Choice: ")
        if ch=="1": add(data)
        elif ch=="2": view(data)
        elif ch=="3": search(data)
        elif ch=="4": update(data)
        elif ch=="5": delete(data)
        elif ch=="6": break

menu()
```

3. Library Management System

```
import json, os
FILE="library.json"

def load(): return json.load(open(FILE)) if os.path.exists(FILE) else []
def save(data): json.dump(data, open(FILE,"w"), indent=4)

def add(data):
    name=input("Book name: "); author=input("Author: ")
    data.append({"name":name,"author":author,"issued":False}); save(data)

def issue(data):
    name=input("Book to issue: "); user=input("User name: ")
    for b in data:
        if b["name"]==name and not b["issued"]:
            b["issued"]=True; b["user"]=user; save(data); return
    print("Not available")

def ret(data):
    name=input("Book to return: ")
    for b in data:
        if b["name"]==name and b["issued"]:
            b["issued"]=False; b.pop("user",None); save(data); return
    print("Not found")

def view(data): [print(b) for b in data if not b["issued"]]
```

```
def menu():
    data=load()
    while True:
        print("\n1.Add 2.Issue 3.Return 4.View Available 5.Exit")
        ch=input("Choice: ")
        if ch=="1": add(data)
        elif ch=="2": issue(data)
        elif ch=="3": ret(data)
        elif ch=="4": view(data)
        elif ch=="5": break

menu()
```

4. Log Analyzer Tool

```
import re

with open("system.log") as f: lines=f.readlines()

ips=re.findall(r"\b\d{1,3}(?:\.\d{1,3}){3}\b",""".join(lines))
dates=re.findall(r"\d{4}-\d{2}-\d{2}",""".join(lines))
errors=[l for l in lines if "ERROR" in l]
ip_count={ip:ips.count(ip) for ip in set(ips)}

summary=f"""
Total Errors: {len(errors)}
IPs: {set(ips)}
Dates: {set(dates)}
Error Lines:
{''.join(errors)}
IP Frequency: {ip_count}
"""

print(summary)
open("summary.txt","w").write(summary)
```

5. Task Automation Tool

```
import os

def list_files(path): [print(f) for f in os.listdir(path)]
def rename_files(path):
    for i,f in enumerate(os.listdir(path),1):
        ext=os.path.splitext(f)[1]
        new=f"new_{i}{ext}"
        os.rename(os.path.join(path,f),os.path.join(path,new))
        print(f"{f} → {new}")
def filter_files(path):
    ext=input("Extension (e.g. .txt): ")
    [print(f) for f in os.listdir(path) if f.endswith(ext)]
```

```
def menu():
    path=input("Enter folder path: ")
    while True:
        print("\n1.List 2.Rename 3.Filter 4.Exit")
        ch=input("Choice: ")
        if ch=="1": list_files(path)
        elif ch=="2": rename_files(path)
        elif ch=="3": filter_files(path)
        elif ch=="4": break

menu()
```